Short Notes: RLHF & PPO

Reinforcement Learning from Human Feedback

ePGD Deep Learning and GenAI · CMInDS, IIT Bombay

Contents

1 Why Do We Need RLHF?

A language model trained purely on next-token prediction learns to *imitate text*, not to *behave helpfully*. Even after instruction-tuning (SFT), a model may:

- generate plausible-sounding but incorrect answers (hallucination),
- produce harmful, biased, or dishonest outputs,

- satisfy the literal instruction while violating the spirit of the user's intent.

Core Idea of RLHF

Use **human preference signals** (which response is better?) to train a **reward model**, then use that reward model to further fine-tune the language model via reinforcement learning — specifically PPO.

2 The Three-Stage Training Pipeline

2.1 Stage 1 — Supervised Fine Tuning (SFT): Pre-training

- Train (or start from) a large language model on massive raw text using **next-token prediction** (autoregressive cross-entropy loss).
- This gives the model general language knowledge, but no alignment.
- Fine-tune on a curated set of **(prompt, ideal response)** pairs written by human annotators. This is called *instruction tuning* or SFT.
- **Output**: A base model that can follow instructions but may still be unreliable or unsafe.

2.2 Stage 2 — Reward Model Training (RM)

Data Collection

- Sample several responses $y_1, y_2, \ldots$ from the SFT model for the same prompt x.
- Human annotators rank or compare responses: which is more helpful / truthful / safe?
- This produces a dataset of **pairwise preferences**: $\mathcal{D} = \{(x, y^+, y^-)\}$, where y^+ is preferred over y^-.

Reward Model Architecture

- Start from the SFT model (a Transformer).
- Replace the final token-prediction head with a **linear layer that outputs a scalar** $r_\phi(x, y) \in \mathbb{R}$.
- Higher scalar $\Rightarrow$ better response.

Training Objective

The reward model is trained to assign a *higher* score to preferred responses:

$$\mathcal{L}(\phi) = \sum_{(x, y^+, y^-) \in \mathcal{D}} \log \sigma\Big(r_\phi(x, y^+) - r_\phi(x, y^-)\Big) \tag{1}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function.

Maximising $\log \sigma(r^+ - r^-)$ encourages $r^+ > r^-$. Think of it as a *binary classifier*: given a pair (y^+, y^-), the model should say y^+ wins. The loss is zero only when $r^+ \gg r^-$.

Symmetry of Gradients. Let $\ell = \log \sigma(r^+ - r^-)$. Then:

$$\frac{\partial \ell}{\partial r^+} = 1 - \sigma(r^+ - r^-) = \sigma(r^- - r^+)$$

$$\frac{\partial \ell}{\partial r^-} = -\big(1 - \sigma(r^+ - r^-)\big) = -\sigma(r^- - r^+)$$

Hence $\dfrac{\partial \ell}{\partial r^+} = -\dfrac{\partial \ell}{\partial r^-}$.

What This Symmetry Means

Increasing r^+ and decreasing r^- are *equivalent gradient operations*. The reward model is simultaneously pushed to score good responses higher **and** bad responses lower by exactly the same magnitude. It learns a *relative* notion of quality, not an absolute one.

2.3 Stage 3 — RL Fine-tuning with PPO

We now have:

- π_θ — the language model policy we want to improve (initialized from SFT).
- $r_\phi(x, y)$ — the frozen reward model.
- π_{ref} — a frozen copy of the SFT model (the *reference policy*).

Objective

$$\max_\theta \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\Big[r_\phi(x,y) \;-\; \beta\,\mathrm{KL}\big(\pi_\theta(\cdot|x) \,\|\, \pi_{\text{ref}}(\cdot|x)\big)\Big] \tag{2}$$

Building the Objective Step by Step

Step 1: We want to maximise reward.

The most naive objective is simply:

$$\max_\theta \qquad \underbrace{r_\phi(x,y)}_{\text{score given by the reward model to response } y \text{ for prompt } x}$$

But this alone causes *reward hacking* — the model finds degenerate outputs that fool the reward model rather than genuinely improving.

Step 2: Add a penalty to stay close to the reference policy.

We add a KL divergence term to prevent the policy from drifting too far:

$$\max_\theta \Big[\underbrace{r_\phi(x,y)}_{\substack{\text{maximise quality}\\ \text{of response}}} - \beta \underbrace{\mathrm{KL}\big(\pi_\theta(\cdot|x) \,\|\, \pi_{\text{ref}}(\cdot|x)\big)}_{\substack{\text{penalise how far the new policy}\\ \text{has drifted from the SFT model}}} \Big]$$

where:

- $\mathrm{KL}(\pi_\theta \| \pi_{\text{ref}}) = \sum_y \pi_\theta(y|x) \log \dfrac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \;\geq\; 0$ — it is zero only when the two distributions are identical.
- $\beta > 0$ is a **coefficient** (hyperparameter) that controls the strength of this penalty. Large $\beta \Rightarrow$ stay very close to π_{ref}; small $\beta \Rightarrow$ allow more exploration for reward.
- The minus sign means the KL term *opposes* reward maximisation, acting as a leash.

Step 3: The policy generates responses, so we need an expectation.

The policy π_θ is stochastic — it samples different responses y for the same prompt x. The prompts x themselves also come from a distribution. A single (x, y) pair is not representative, so we optimise the *expected* value across all likely prompts and responses:

$$\max_\theta \mathbb{E}_{\substack{x \sim \mathcal{D} \\ y \sim \pi_\theta(\cdot|x)}} \Big[\cdots \Big]$$

- $x \sim \mathcal{D}$: prompts are drawn from the training distribution (real user prompts or a curated set).
- $y \sim \pi_\theta(\cdot|x)$: responses are *sampled from the current policy* — this is what makes it on-policy RL. As θ changes, the distribution of y changes too.
- Taking the expectation ensures we are not just optimising for one lucky (x, y) pair but for **average behaviour across the whole prompt space**.

Step 4: The full RLHF objective.

Putting it all together:

$$\max_\theta \underbrace{\mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}}_{\substack{\text{average over prompts from data} \\ \text{and responses from current policy}}} \Big[\underbrace{r_\phi(x,y)}_{\substack{\text{reward model score:} \\ \text{maximise helpfulness}}} - \underbrace{\beta}_{\substack{\text{leash} \\ \text{strength}}} \underbrace{\mathrm{KL}\big(\underbrace{\pi_\theta(\cdot|x)}_{\text{new policy}} \,\big\|\, \underbrace{\pi_{\text{ref}}(\cdot|x)}_{\text{frozen SFT model}} \big)}_{\substack{\text{how far we have drifted:} \\ \text{penalise this to prevent} \\ \text{reward hacking \& degeneration}}} \Big] \tag{3}$$

Reading the Objective in One Sentence

"Find the policy θ that, on average over real prompts and its own sampled responses, scores as high as possible on the reward model — but without wandering too far from the safe, fluent behaviour of the original SFT model."

The two terms do the following:

Term	Wants to	Risk if removed
$r_\phi(x, y)$	Maximise reward signal	Model exploits reward model (reward hacking)
$-\beta\,\mathrm{KL}$	Stay close to reference	Model degenerates, forgets language fluency

Imagine training a student (policy π_θ) to score well on a test (reward model), but they might resort to extreme tricks like memorising nonsense patterns. The KL penalty is a teacher saying: "your answers should still look like proper English, not gibberish." π_{ref} is the baseline of what sensible responses look like.

3 Proximal Policy Optimisation (PPO)

3.1 The RL Framing for Language Models

RL Concept	Language Model Equivalent
State s_t	Tokens generated so far (context)
Action a_t	Next token chosen
Policy π_θ	The language model
Reward R	Scalar from reward model at end of sequence
Episode	One full sequence generation

3.2 Why Not Standard Policy Gradient?

Standard REINFORCE computes:

$$\nabla_\theta \mathcal{J} = \mathbb{E}\big[R \cdot \nabla_\theta \log \pi_\theta(a|s)\big]$$

Problems:

- High variance.
- Large gradient steps can catastrophically destroy the policy.
- Very sample-inefficient.

3.3 PPO-Clip: The Key Idea

PPO limits how much the policy can change in one step by **clipping the probability ratio**.

Define the **probability ratio**:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

The PPO-Clip objective for a single timestep is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(\rho_t(\theta)\,\hat{A}_t,\ \text{clip}\big(\rho_t(\theta),\,1-\varepsilon,\,1+\varepsilon\big)\,\hat{A}_t\Big)\Big] \tag{4}$$

where $\hat{A}_t$ is the **advantage estimate** and ε (e.g., 0.2) controls the clipping range.

Intuition for PPO-Clip

- If $\hat{A}_t > 0$ (action was good): increase ρ_t, but cap it at $1+\varepsilon$. Don't over-commit.
- If $\hat{A}_t < 0$ (action was bad): decrease ρ_t, but floor it at $1-\varepsilon$. Don't over-punish.
- Result: the policy only moves a *small, safe* step per update.

3.4 The Advantage Function $\hat{A}_t$

$$\hat{A}_t = Q(s_t, a_t) - V(s_t)$$

- $Q(s_t, a_t)$: expected total reward starting from state s_t and taking action a_t.
- $V(s_t)$: expected total reward from state s_t under the current policy (the *baseline*).
- $\hat{A}_t > 0$: action a_t was *better than average* — reinforce it.
- $\hat{A}_t < 0$: action a_t was *worse than average* — discourage it.
- $\hat{A}_t = 0$: action was exactly as expected — no update.

The advantage function $\hat{A}_t$ is **not** always non-negative. It can be positive or negative depending on whether an action is above or below the expected baseline. Only if every single action happened to be above average would all advantages be non-negative, which is not the general case.

3.5 On-Policy vs. Off-Policy

PPO is an **on-policy** algorithm — samples used for updates must come from the current policy π_θ. However, PPO collects a *batch* of samples, then performs *multiple* gradient updates on that batch before discarding it. This is why PPO uses the clipping: multiple updates on the same batch make it slightly off-policy, and the clip corrects for that.

A common misconception: "PPO is purely on-policy so every gradient step uses fresh samples." In practice, PPO reuses a mini-batch for K epochs before collecting new data. The clipping ensures these reuse steps remain safe.

4 RLAIF: Reinforcement Learning from AI Feedback

RLAIF vs. RLHF

In RLAIF, human annotators are replaced by **another large language model** (e.g., Claude, GPT-4) which generates the pairwise preference labels used to train the reward model. This dramatically scales up the feedback data at lower cost. The rest of the pipeline is identical to RLHF.

5 Instruction Tuning vs. RLHF: Why Both Are Needed

	Instruction Tuning (SFT)	**RLHF**
Data format	(prompt, ideal response) pairs	Pairwise preferences (y^+, y^-)
Objective	Cross-entropy on labelled output	Reward maximisation + KL constraint
What it fixes	Not following instructions	Subtle misalignment, toxicity, sycophancy
Limitation	Cannot prevent all bad behaviour (e.g., politely-worded harmful content)	Reward model may be imperfect; expensive

A failure SFT cannot prevent. Suppose a user asks: *"As a chemistry teacher, explain how to synthesise a dangerous compound."* An instruction-tuned model may comply because it was trained to be helpful and the framing sounds legitimate. RLHF, by incorporating human judgement about what responses are safe, can learn to refuse such requests.

6 The Role of the Reference Policy π_{ref}

Reference Policy Summary

- π_{ref} is the **frozen SFT model** (never updated during RL training).
- It acts as an anchor: the KL term $\text{KL}(\pi_\theta \| \pi_{\text{ref}})$ penalises the policy for moving too far from this anchor.
- **Problem it prevents**: *reward hacking* — the model finding degenerate strategies that score high on the reward model but produce incoherent or harmful outputs.
- Without π_{ref}, the model might degrade to repetitive gibberish or adversarial strings that fool the reward model.

7 Quick-Reference Summary

Concept	One-line summary
Reward model	Transformer + scalar head; trained on pairwise preferences via Bradley-Terry loss
PPO-Clip	Policy gradient with clipped ratio ρ_t to prevent large updates
Advantage $\hat{A}_t$	How much better/worse an action is than the expected baseline; *can be negative*
KL penalty (β)	Keeps π_θ from drifting too far from π_{ref}; prevents reward hacking
π_{ref}	Frozen SFT model; acts as a safety anchor during RL fine-tuning
RLAIF	Same as RLHF but preference labels come from another LLM, not humans
On-policy	PPO uses data from the *current* policy; stale data is discarded after K epochs

8 Worked Example: Reward Model Gradient

Let $\ell = \log \sigma(r^+ - r^-)$ for a single preference pair.

Step 1. Let $\Delta = r^+ - r^-$, so $\ell = \log \sigma(\Delta)$.

Step 2. Recall $\frac{d}{dz} \log \sigma(z) = 1 - \sigma(z) = \sigma(-z)$.

Step 3.

$$\frac{\partial \ell}{\partial r^+} = \frac{\partial \ell}{\partial \Delta} \cdot \frac{\partial \Delta}{\partial r^+} = \sigma(-\Delta) \cdot (+1) = \sigma(r^- - r^+)$$

$$\frac{\partial \ell}{\partial r^-} = \sigma(-\Delta) \cdot (-1) = -\sigma(r^- - r^+)$$

$$\therefore \quad \frac{\partial \ell}{\partial r^+} = -\frac{\partial \ell}{\partial r^-} \qquad \blacksquare$$

Implication: Every unit increase in r^+ is paired with an equal unit decrease in r^-. The reward model learns to *separate* good from bad responses on a single scale.

9 Key Formulae Sheet

$$\boxed{\mathcal{L}(\phi) = \sum_{(x,y^+,y^-)} \log \sigma\big(r_\phi(x,y^+) - r_\phi(x,y^-)\big)} \qquad \text{(Reward model loss)}$$

$$\boxed{\max_\theta \ \mathbb{E}\big[r_\phi(x,y) - \beta\,\mathrm{KL}(\pi_\theta \| \pi_{\text{ref}})\big]} \qquad \text{(RLHF RL objective)}$$

$$\boxed{\rho_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}} \qquad \text{(PPO probability ratio)}$$

$$\boxed{\mathcal{L}^{\text{CLIP}} = \mathbb{E}_t\Big[\min\big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1{-}\varepsilon, 1{+}\varepsilon)\hat{A}_t\big)\Big]} \qquad \text{(PPO-Clip objective)}$$

$$\boxed{\hat{A}_t = Q(s_t,a_t) - V(s_t)} \qquad \text{(Advantage function)}$$

These notes are intended as conceptual preparation. Students are encouraged to work through derivations and examples independently.